GhostLogin - Automated SSH Exposure and Attack

Project Overview -

The project is a fully automated tool designed to detect hosts running SSH services within a defined internal subnet range.

The tool validates user input, scans the network for exposed SSH services, performs controlled authentication testing, and executes predefined verification commands without establishing interactive sessions.

By minimizing user interaction and automating all operational stages, the system ensures efficiency, consistency, and controlled security validation.

Creativity -

One of the improvements added to the project is an automatic dependency checker.
The script verifies that required tools such as hydra, nmap, and sshpass are installed before execution. If a tool is missing, the user is asked whether to install it. This prevents the script from failing during runtime.

Another improvement is a smart password list management logic.
The script searches automatically for the rockyou.txt file in common system directories. If the file is not found, the script offers to download it. This prevents the program from failing due to a missing wordlist and makes the tool more flexible.

In addition, instead of executing a simple verification command, the script collects extended system and environment information such as OS version, uptime, geographic location, and service provider.
This transforms the post-access phase into an automated information gathering stage rather than a simple login confirmation.

Conclusion -

GhostLogin successfully automates the SSH assessment process.
The project demonstrates automation, controlled credential testing, and post-access information gathering in a structured and safe manner.

Operational Logic Flow -

Necessary Tools Check:
The script checks if hydra, nmap, and sshpass tools are installed.
If a tool is missing, the program asks the user if they want to install it. If they choose not to install the tool, the program will terminate.

Finding The Targets:
The script automatically finds the user's local IP to identify the user's inner network IP range. With this information, the script continues the process in the later stages of the code.

Asking For Subnet Mask:
The script asks the user to enter a subnet mask and recommends using 24.
After that, it checks that the value entered is valid. If the value is not valid, the script asks again until a correct subnet mask is entered.

Scanning:
The script takes the IP range and runs an nmap scan to identify machines in the user's inner network, specifically looking for machines with port 22 (SSH) open.
The IP addresses that have SSH open are saved into a list of targets.

Picking The Password File:
The script looks for the rockyou.txt password file on the user's system using the "find" command in common folders (such as /usr/share, /home, /root, etc.), because "locate" might be slower and rockyou.txt might not be in its regular path.
If the user does not have the rockyou.txt file, the program suggests downloading it or using a built-in default password list instead.

Initiating Attack Attempt:
The script goes through every host it found and tries to log in using common usernames (like root, admin, user) and also the current user (whoami).
It uses hydra to perform the password attempts automatically in the background.

Gathering User Information:
Once the script finds working credentials, it uses sshpass to automatically collect information such as username, OS version, country, city, uptime, service provider, and local time. The information is then displayed to the user.

Automation:
The script bypasses the host key verification prompt. This means the user does not need to interact with the SSH session, except for entering the subnet mask value.
The script also uses connection timeout settings to prevent hanging SSH sessions in case of unreachable hosts.

Final Report:
At the end, the user receives a clear "Final Session Report" with the list of all the IPs, hosts, usernames, and passwords that were found, and exactly which ones were successfully accessed.

Objectives Implementation -

User Input and Validation:
The script automatically detects the local network IP using ip route.
After that, the user is asked to enter a subnet mask between 16-32.
The value is cleaned from spaces and checked to make sure it is numeric and inside the allowed range.
If the value is invalid, the script asks again until a valid subnet mask is entered.

Scanning for SSH Services:
The script builds the IP range in CIDR format and runs a nmap scan only on port 22.
Only machines with SSH open are saved into the ssh_array.
If no SSH services are found, the attack phase does not start.

Credential Testing:
The script searches for rockyou.txt in common system directories using the find command. If found, only the first 1100 passwords are extracted into a temporary file in /tmp to limit system load and keep the process controlled. If the file is not found, the script offers to download it. If declined, a default password list is used.
When using the built-in list, a temporary password file is created in /tmp, since Hydra requires a local password file via the -P parameter.
The script attempts authentication using common usernames (whoami, root, admin, user).
Hydra runs in non-interactive mode and stops after the first successful password per host.
All temporary password files are deleted after execution to maintain system cleanliness.

Post-Access Automation:
When valid credentials are found, the script connects using sshpass and runs predefined commands automatically.
It bypasses host key verification and uses a connection timeout to prevent hanging sessions. No interactive shell is opened.
The script collects system details such as username, OS version, country, city, uptime, service provider, and local time.

Output and Reporting:
At the end, the script generates a clear “Final Session Report”.
It displays all hosts with open SSH ports and shows which ones were successfully accessed, including the IP, username, and password that were found.

Output Screenshot –

The screenshot demonstrates the complete automated execution flow of the tool, including dependency validation, local IP detection, subnet validation, SSH scanning, credential testing, automated information gathering, and final session reporting.
The execution is performed without manual SSH interaction, demonstrating full automation.

